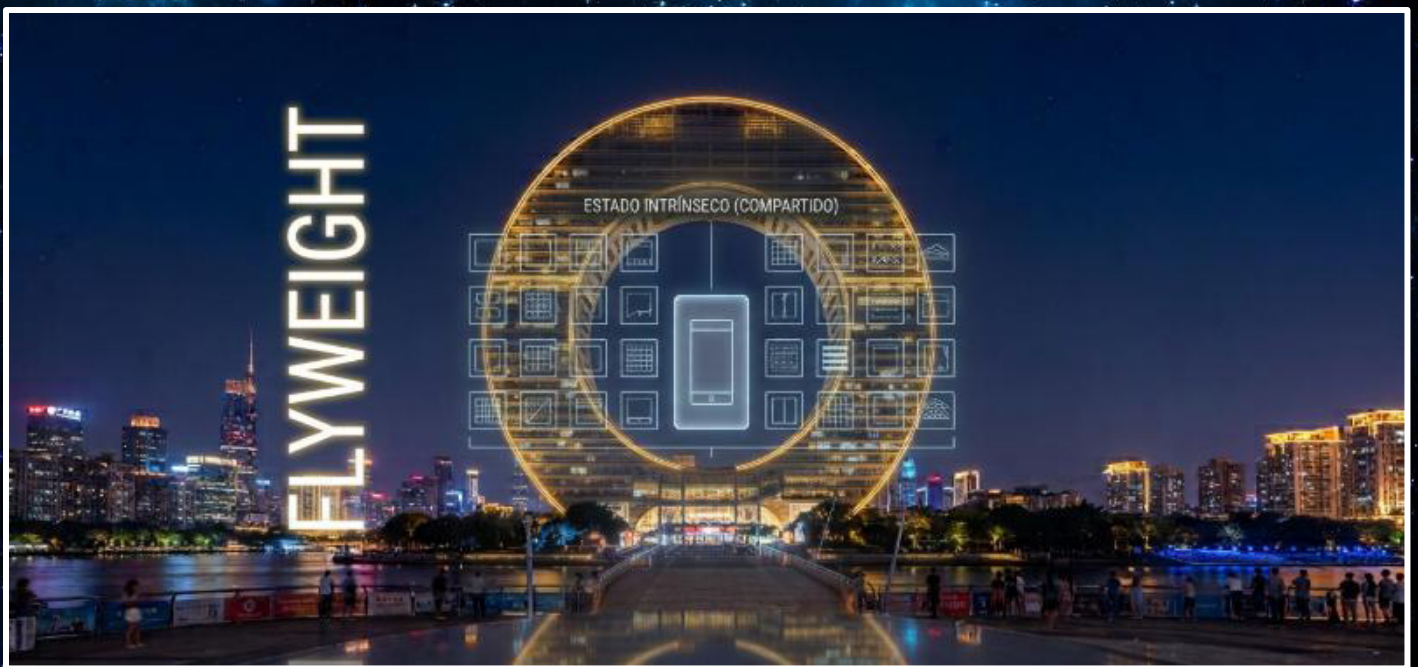


Patrones de Diseño

Estructurales



Teoría y práctica aplicada con Java

Yim APESTEGUI

Yim Isaias Apestegui Florentino

Patrones de diseño estructurales

Patrones de diseño estructurales

Autor – Editor:

© Yim Isaias Apestegui Florentino

Garatea Mz.100 Lote 7 – Nuevo Chimbote

yapestegui@gmail.com

Ancash - Perú

Primera edición digital, setiembre 2026

Depósito Legal N° 2026-10464

ISBN:978-612-03-3119-4

ISBN: 978-612-03-3119-4



Libro disponible en: <https://yimaf.github.io/books/pds-estructurales.html>

Tabla de contenido

Prólogo.....	vi
1 Introducción a los patrones de diseño estructurales	1
1.1 ¿Qué son los patrones de diseño?.....	1
1.2 Clasificación general de los patrones GoF	1
1.3 Conceptos previos indispensables.....	2
1.4 Dependencia de la Abstracción.....	4
1.5 Introducción al Desacoplamiento Total (Polimorfismo).....	4
1.6 Inyección de Dependencias	5
1.7 El "problema raíz"	11
1.8 Variantes concretas a considerar	11
2 Adapter (Adaptador).....	12
2.1 Venta de comida en una cevichería	12
2.2 Diagrama UML Adapter	12
2.3 Código Adapter	13
3 Bridge (Separador)	15
3.1 Comida peruana y canales de atención.....	15
3.2 Diagrama UML Bridge	15
3.3 Código Bridge	16
4 Composite (Compuesto).....	19
4.1 Pedido a la Carta en una cevichería.....	19
4.2 Diagrama UML Composite	20
4.3 Código Composite.....	20
5 Decorator (Decorador).....	23
5.1 Venta de anticucho con complementos	23

5.2	Diagrama UML Decorator.....	23
5.3	Código Decorator.....	24
6	Facade (Fachada).....	27
6.1	Organizar un viaje turístico a Cusco.....	27
6.2	Diagrama UML Facade	27
6.3	Código Facade	28
7	Flyweight (Peso ligero).....	30
7.1	Mapa turístico del Perú	30
7.2	Diagrama UML Flyweight.....	30
7.3	Código Flyweight	30
8	Proxy (representante).....	33
8.1	Acceso a información turística.....	33
8.2	Diagrama UML Proxy.....	33
8.3	Código Proxy.....	33
9	Comparación de Patrones de Diseño Estructurales	36
9.1	Similitudes transversales	38
10	Perspectiva moderna de los patrones de diseño estructurales	39
10.1	Limitaciones del patrón de diseño.....	39
10.2	Patrones estructurales en arquitecturas modernas.....	39
10.3	Cuándo NO usar patrones estructurales (principio YAGNI)	39
10.4	Tabla de decisión final	40
11	Epílogo	41
	Bibliografía	42

Prólogo

Escribir software no se trata solo de codificar y hacer que el código funcione; se trata de lograr que sobreviva al tiempo. Cuando empezamos a programar, la emoción de ver un "Hola Mundo" o de probar y compilar un sistema funcional nos hace creer que vamos por buen camino, dado que nuestras primeras pruebas funcionan. Sin embargo, la verdadera prueba llega un tiempo después, cuando el cliente pide un cambio, las reglas de negocio se transforman o el sistema necesita crecer y las cosas se ponen inmanejables.

Es ahí donde el código puro y rígido se rompe y donde los desarrolladores pasamos noches enteras descifrando lo que otros escribieron o lo que nosotros mismos escribimos. El objetivo de este libro es tender un enlace entre ese código que "funciona por ahora bien" y el software escalable y fácil de mantener. A través de estas páginas, exploraremos los patrones de diseño estructurales, que son soluciones probadas y refinadas por la comunidad a lo largo de décadas para resolver problemas comunes en la arquitectura de software.

Pensado especialmente para estudiantes de ingeniería de sistemas, sistemas e informática, ciencias de la computación y desarrolladores junior que ya dominan la sintaxis básica y buscan subir el siguiente peldaño en su carrera. Para aprovechar al máximo este contenido, solo necesitas tener conocimientos previos en Java, específicamente en los pilares de la Programación Orientada a Objetos: clases, interfaces, herencia, polimorfismo y el manejo básico de colecciones. De hecho, no necesitas ser un experto en arquitectura, porque aquí construiremos esa base paso a paso.

Para aterrizar la teoría abstracta y alejarla de los aburridos ejemplos genéricos de la buena literatura anglosajona, cada patrón estructural será explicado utilizando escenarios y problemáticas del día a día en el entorno peruano que fácilmente puede aplicar a Iberoamérica, américa latina y el

caribe. Comenzaremos explorando el patrón Adapter, diseñado para conectar interfaces incompatibles. Lo entenderemos resolviendo un caso muy común: cómo integrar el sistema de pagos de una pequeña empresa pymes como una cevichería con un software de facturación que ya ha venido funcionando que el negocio no puede cambiar. Luego, estudiaremos el patrón Bridge, cuya misión es separar dos dimensiones de un problema que pueden variar de forma independiente. Para este caso, modelaremos la preparación de platos tradicionales peruanos y cómo estos se conectan con sus diferentes canales de atención, ya sea para consumo en salón, delivery o recojo en tienda.

Seguiremos con el patrón Composite, ideal para manejar estructuras jerárquicas en forma de árbol de manera unificada. La mejor forma de visualizarlo será construyendo de forma lógica la carta completa de una cevichería, organizando dinámicamente desde combos familiares hasta platos individuales, bebidas y guarniciones. Posteriormente, veremos el patrón Decorator, que nos permite agregar funcionalidades de manera dinámica a un objeto sin alterar su estructura. Lo aplicaremos en una carretilla digital de anticuchos, permitiendo al usuario personalizar su pedido añadiéndole complementos y salsas en tiempo de ejecución de manera totalmente flexible.

Para los sistemas que se vuelven demasiado grandes, el patrón Facade nos enseñará a simplificar la interacción con subsistemas complejos. Lo implementaremos creando una interfaz única y amigable para gestionar un paquete turístico completo a Cusco, unificando procesos internos tan distintos como la reserva de vuelos, hoteles y guías de turismo en un solo clic. Con el patrón Flyweight, reduciremos el uso de memoria compartiendo objetos masivos y repetidos, recreando un mapa digital que debe renderizar miles de iconos de lugares turísticos nacionales sin ralentizar la aplicación.

Luego, utilizaremos el patrón Proxy como un intermediario que controla, protege y cachea el acceso a un servicio pesado de información turística de alta demanda.

Se recomienda para poder replicar cada caso usar una IDE de Java de su preferencia como: Netbeans, Visual Studio Code, IntelliJ IDEA o Eclipse, luego llevar el código a su espacio de codificación. Puede copiar y pegar, aunque fuertemente no lo recomiendo por ahora, sino mejor reescribir el código, de esa manera errores que vayan apareciendo tendrán sentido resolverlos y además no perderá la satisfacción de poderlos resolver. Claro está que el código está probado y si Ud. copia y pega no tendrá la experiencia de aprender de los errores en codificar.

Se presenta para cada caso planteado un diagrama de clases UML, a fin de perfeccionar su comprensión del patrón estructural y de que Ud. pueda comprobar la coherencia entre el código y el diseño de clases UML.

Con este mapa de ruta, cada tema te llevará de la teoría a un código Java real y su respectivo diseño de clases UML, que podrás reconocer en cualquier negocio de tu país, dándote las herramientas necesarias para diseñar software como un verdadero profesional.

1 Introducción a los patrones de diseño estructurales

1.1 ¿Qué son los patrones de diseño?

Un patrón de diseño es una solución que se aplica a un problema recurrente dentro de un contexto de diseño de software. No se trata solo de código, porque en realidad cualquier patrón de diseño no debería estar sujeto a algún lenguaje de programación, sino que es como una plantilla aplicable a distintas situaciones que ya otros tuvieron la oportunidad de atravesar. Los patrones documentan la experiencia colectiva de diseñadores expertos, capturando "el problema, la solución, cuándo aplicarla y sus consecuencias" (Gamma et al., 1994).

1.2 Clasificación general de los patrones GoF

Hay un libro que se menciona bastante como referencia: El libro fundacional de Gamma, Helm, Johnson y Vlissides (conocidos como la "Gang of Four" o GoF), el cual clasifica los patrones en tres categorías según su **propósito**:

Categoría	Propósito	Pregunta que responde
Creacionales	Cómo se instancian los objetos	¿Cómo se crea un objeto?
Estructurales	Cómo se componen clases y objetos	¿Cómo se ensamblan las piezas?
De comportamiento	Cómo interactúan y se distribuyen responsabilidades	¿Cómo se comunican los objetos?

Los patrones estructurales específicamente responden:

"¿cómo organizo clases y objetos existentes en estructuras más grandes, manteniendo el sistema flexible y eficiente?" (Gamma et al., 1994, cap. 4).

1.3 Conceptos previos indispensables

Para entender los patrones estructurales, es necesario dominar estos pilares:

a) Composición vs. Herencia

- **Herencia:** es una relación estática que se entiende como “es un”, definida en tiempo de compilación.
- **Composición:** es una relación dinámica entendida como "tiene un", definida en tiempo de ejecución mediante referencias a objetos.

Los patrones estructurales generalmente permiten la composición porque es más flexible, es decir: permite cambiar el comportamiento en tiempo de ejecución sin crear jerarquías de clases rígidas ni sufrir la famosa "explosión combinatoria de subclases".

Veamos los siguientes ejemplos de fragmento de código:

Herencia (relación estática "es un")	PROBLEMA
<pre>// Herencia: rígida, definida en tiempo de compilación class Impresora { public void imprimir(String texto) { System.out.println("Imprimiendo: " + texto); } } class ImpresoraConEscaner extends Impresora { public void escanear() { System.out.println("Escaneando documento..."); } }</pre>	Si mañana necesitamos una ImpresoraConFax, o una ImpresoraConEscaner YFax, la jerarquía de clases crece exponencialmente (explosión combinatoria).

Composición (relación dinámica "tiene un")	MEJORA
<pre>// Composición: flexible, definida en tiempo de ejecución interface Escaner { void escanear(); } class EscanerLaser implements Escaner { public void escanear() {</pre>	Ahora se puede inyectar cualquier implementación de Escaner sin tocar la clase Impresora. Esta es la base de todos

<pre> System.out.println("Escaneando con láser..."); } } class Impresora { private Escaner; // composición: la impresora TIENE un escáner public Impresora(Escaner escaner) { this.escaner = escaner; } public void imprimir(String texto) { System.out.println("Imprimiendo: " + texto); } public void escanear() { escaner.escanear(); // delegación } } </pre>	los patrones estructurales.
---	--

b) Programar siempre hacia una interfaz, no hacia una implementación

La POO permite que distintos objetos sean intercambiables si cumplen el mismo contrato (interfaz). De esa situación se apoyan un Proxy o un Decorator; sustituyen a un objeto real sin que el cliente siquiera lo note (Gamma et al., 1994).

Veamos el siguiente fragmento de código:

App.java
<pre> interface MedioDePago { void pagar(double monto); } class PagoTarjeta implements MedioDePago { public void pagar(double monto) { System.out.println("Pagando \$" + monto + " con tarjeta"); } } class PagoYape implements MedioDePago { public void pagar(double monto) { System.out.println("Pagando \$" + monto + " con Yape"); } } class Carrito { // El cliente depende de la ABSTRACCIÓN, no de la implementación concreta private MedioDePago medioDePago; </pre>

```
public Carrito(MedioDePago medioDePago) {  
    this.medioDePago = medioDePago;  
}  
public void finalizarCompra(double total) {  
    medioDePago.pagar(total);  
}  
}
```

1.4 Dependencia de la Abstracción

Respecto al código anterior, para entenderlo de forma muy sencilla, imagina esta analogía de la vida real: cuando vas a un supermercado, por ejemplo Plaza Vea, Tottus, Saga Falabella, Mass entre otros el cajero te dice el importe total a pagar y Ud. Observa un **terminal de pago**. A ti no te importa si ese aparato por dentro procesa una tarjeta Visa, una Mastercard, un chip sin contacto o un código QR; tú solo sabes que eliges tu método de pago ahí y el dinero se transfiere. Ese terminal es la **abstracción**. Por eso, el cliente depende de la Abstracción.

El Carrito está blindado contra cambios (private MedioDePago medioDePago) El carrito no dice private PagoTarjeta medioDePago; ni private PagoYape medioDePago;. Si mañana decides agregar Plin, PagoEfectivo o Apple Pay, no tienes que modificar ni una sola línea de código dentro de la clase Carrito. Solo creas una nueva clase que implemente MedioDePago y listo. El carrito seguirá funcionando exactamente igual.

1.5 Introducción al Desacoplamiento Total (Polimorfismo)

Cuando el carrito ejecuta la acción: medioDePago.pagar(total);

Carrito no sabe si se va a ejecutar el System.out.println de la PagoTarjeta o el de PagoYape. **Eso se decide en tiempo de ejecución gracias al polimorfismo** (Eckel, 2007). El Carrito confía ciegamente en que cualquier objeto que reciba por el constructor tendrá un método llamado pagar() que acepta un double como parámetro.

1.6 Inyección de Dependencias

Al usar el constructor:

```
public Carrito(MedioDePago medioDePago) {
    this.medioDePago = medioDePago;}
```

Le estás quitando al Carrito la responsabilidad de crear el método de pago. Si dependiera de la implementación concreta (NO SUGERIDO): Harías un `new PagoYape()` dentro del carrito, amarrando el carrito a Yape para siempre. Al depender de la abstracción (BUENO): El "cliente" que use el carrito afuera decidirá dinámicamente qué pasarle:

```
new Carrito(new PagoYape()) ó new Carrito(new PagoTarjeta()).
```

Este término “Inyección de dependencias” fue establecido por (Fowler, 2004).

c) Acoplamiento y cohesión

En un **Acoplamiento bajo** los módulos dependen poco unos de otros (es lo deseable). En una **Cohesión alta** cada módulo tiene una responsabilidad bien definida (es lo deseable). Los patrones estructurales existen, para eso, para **reducir el acoplamiento** entre subsistemas, clientes e implementaciones concretas (Larman, 2003).

Veamos el siguiente código:

ALTO ACOPLAMIENTO	BAJO ACOPLAMIENTO
<pre>// Mal diseño: el cliente conoce las clases concretas class ReporteMal { private GeneradorPDF pdf = new GeneradorPDF(); // dependencia rígida public void generar() { pdf.crear(); } }</pre>	<pre>//Buen diseño: el cliente conoce solo la interfaz interface GeneradorDocumento { void crear(); } class ReporteBien { private GeneradorDocumento generador; // dependencia hacia abstracción public ReporteBien(GeneradorDocumento generador) { this.generador = generador; } public void generar() { generador.crear(); } }</pre>

En realidad no es muy conveniente que un cliente conozca cómo trabaja una clase concreta, aunque a veces y en ocasiones es necesario, pero ello genera fuerte acoplamiento. En contraste, un cliente debería conocer solo a la interfaz. La clase ReporteBien puede trabajar con cualquier tipo de GeneradorDocumento (PDF, Word, HTML) y lo más interesante; sin modificarse. Esto es precisamente lo que hacen Bridge y Adapter

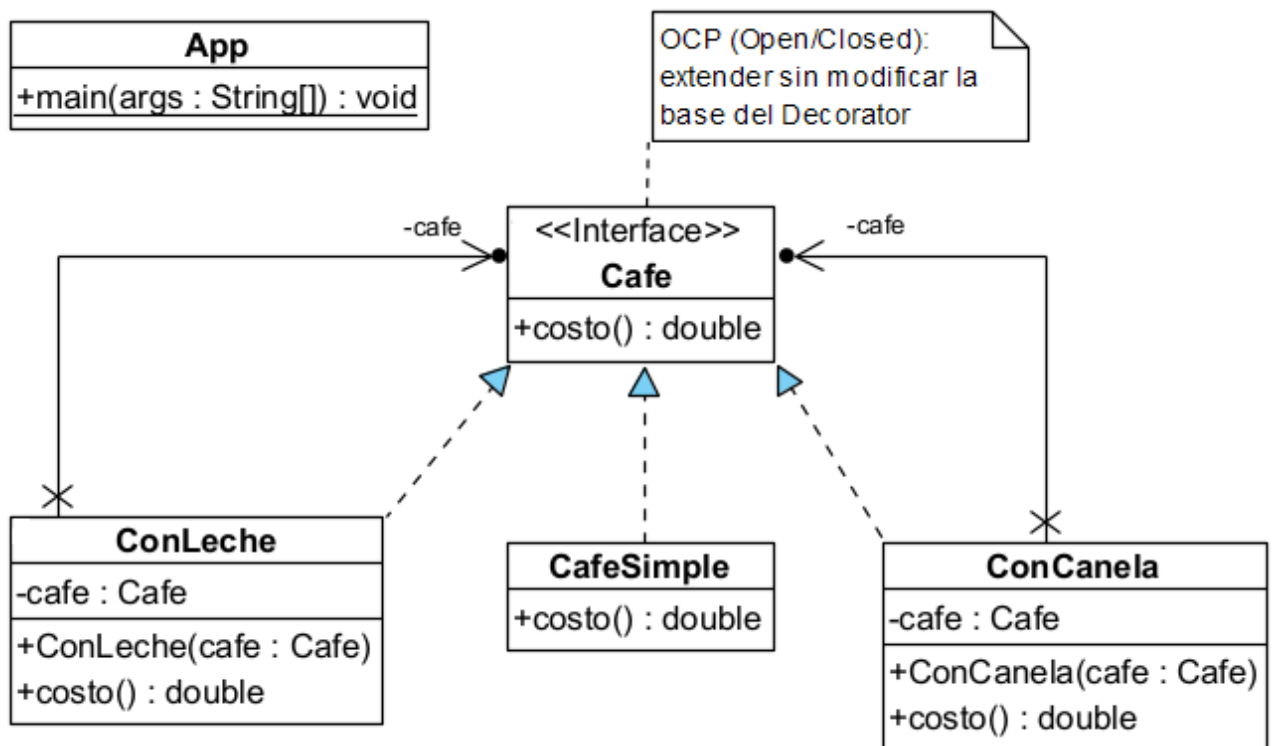
d) Principios SOLID relevantes

Aunque los principios SOLID, corresponden verse más en el tema de Arquitectura de Software, se hace necesario tener la noción de estos principios dado que están muy relacionados con los patrones estructurales.

Principio	Relación con patrones estructurales
Single Responsibility (SRP)	Con Facade pues separa la complejidad del subsistema de la lógica del cliente
Open/Closed (OCP)	Con Decorator quien permite extender comportamiento sin modificar la clase original
Liskov Substitution (LSP)	Relacionado con Adapter y Proxy pues dependen de que los objetos sean sustituibles por su interfaz
Dependency Inversion (DIP)	Con Bridge quien separa abstracciones de implementaciones concretas mediante interfaces

Fuente: (Martin, 2000)

Veamos el siguiente caso para entender un componente del principio SOLID, que tiene que ver con el principio OCP (Abierto a la extensión y cerrado a la modificación). Un CafeSimple puede ser decorado con leche o con canela



En este caso ya estamos frente a un Decorator, diríamos completo y muy funcional: cada envoltorio añade un comportamiento (costo) sin modificar la clase original CafeSimple, cumpliendo el principio OCP (Martin, 2000).

```

App.java

// OCP (Open/Closed): extender sin modificar base del Decorator
interface Cafe {
    double costo();
}

class CafeSimple implements Cafe {
    public double costo() { return 5.0; }
}

class ConLeche implements Cafe {
    private Cafe cafe; // composición, NO herencia

    public ConLeche(Cafe cafe) {
        this.cafe = cafe;
    }

    public double costo() {
        return cafe.costo() + 1.5; // extiende sin modificar CafeSimple
    }
}

class ConCanela implements Cafe {
    private Cafe cafe;

    public ConCanela(Cafe cafe) {

```

```

        this.cafe = cafe;
    }

    public double costo() {
        return cafe.costo() + 0.5;
    }
}

public class App {
    public static void main(String[] args) {
        Cafe pedido = new ConCanela(new ConLeche(new CafeSimple()));
        System.out.println("Total: $" + pedido.costo()); // salida
esperada 7.0
    }
}

```

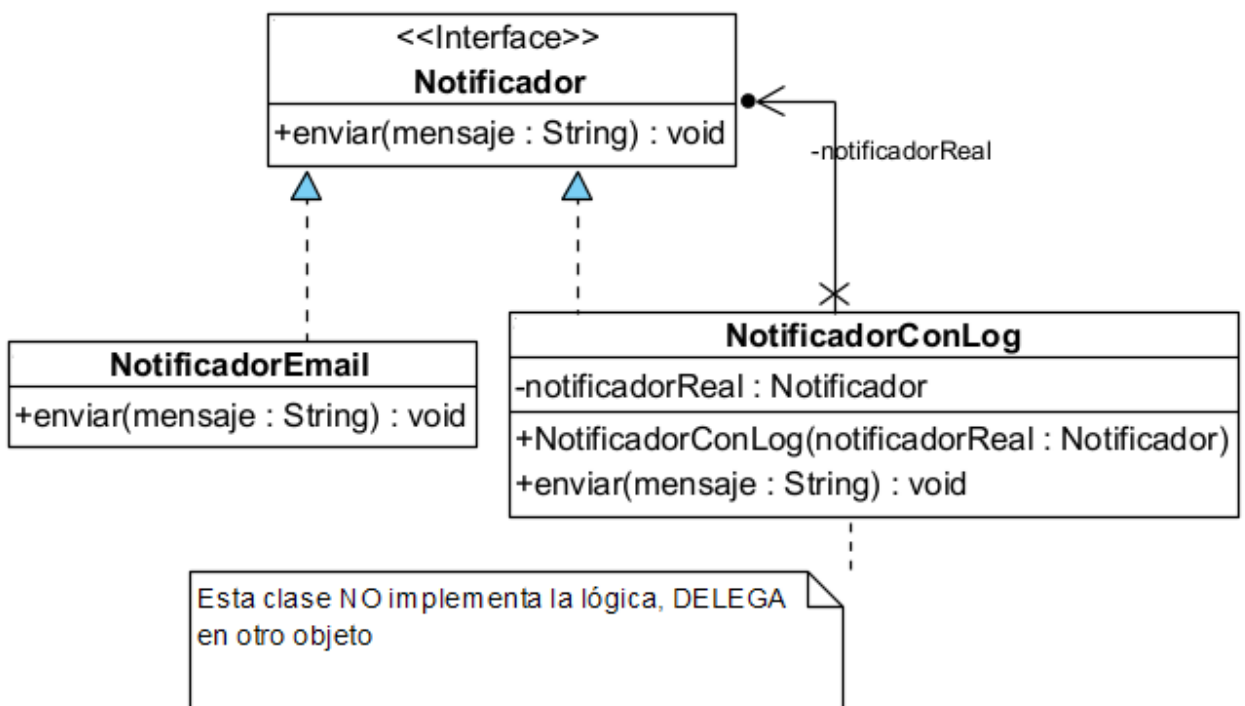
Salida:

Total: \$7.0

e) Delegación

Es un mecanismo mediante el cual un objeto reenvía (delega) una solicitud a otro objeto en lugar de resolverla él mismo. Es el mecanismo de ejecución interno de **todos** los patrones estructurales, por ejemplo: Adapter delega en el adaptado, Proxy delega en el objeto real, Decorator delega en el componente envuelto, etc. (Gamma et al., 1994).

Veamos el siguiente Diagrama de Clases



El constructor público NotificadorConLog referencia a Notificador, a través del objeto notificadorReal, por eso es que esta clase aunque debería ser un implementador, NO IMPLEMENTA, la lógica, sino la DELEGA,

Leer el siguiente fragmento de código:

App.java

```
interface Notificador {
    void enviar(String mensaje);
}

class NotificadorEmail implements Notificador {
    public void enviar(String mensaje) {
        System.out.println("Email enviado: " + mensaje);
    }
}

// Esta clase NO implementa la lógica, DELEGA en otro objeto
class NotificadorConLog implements Notificador {
    private Notificador notificadorReal; // referencia al objeto delegado

    public NotificadorConLog(Notificador notificadorReal) {
        this.notificadorReal = notificadorReal;
    }

    public void enviar(String mensaje) {
        System.out.println("[LOG] Intentando enviar mensaje...");
        notificadorReal.enviar(mensaje); // delegación
        System.out.println("[LOG] Mensaje enviado con éxito");
    }
}
```

Como se puede notar en el anterior fragmento de código: la clase envolvente implementa la misma interfaz que el objeto envuelto, pero mantiene una referencia a ese objeto. En otras palabras delega la llamada, añadiendo comportamiento antes/después. Como aun no hemos visto el patrón estructural respectivo, déjame decirte que este modelo se parece mucho a algún tipo de Decorador, que veremos más adelante.

f) Estructura de objetos vs. estructura de clases

En esta sección vamos a analizar un poco más la forma en que se componen los patrones estructurales según Gamma et al. (1994).

- Los **Patrones estructurales de clase**: usan herencia para componer interfaces o implementaciones (ej. Adapter de clase).
- Los **Patrones estructurales de objeto**: usan composición de objetos en tiempo de ejecución (la mayoría de los casos prácticos: Decorator, Composite, Proxy, Facade, Bridge, Flyweight).

Mediante Herencia	Mediante Composición
Patrón de Clase que es muy poco usado	Patrón de objeto que es el más usado
<pre> class Rectangulo { public void dibujar() { System.out.println("Dibujando rectángulo"); } } // Adapter de clase: usa herencia múltiple de interfaces (Java lo simula con interfaces) interface Forma { void render(); } class AdaptadorRectangulo extends Rectangulo implements Forma { public void render() { dibujar(); // reutiliza el método heredado } } </pre>	<pre> // Adapter de objeto: composición en lugar de herencia class Rectangulo { public void dibujar() { System.out.println("Dibujando rectángulo"); } } interface Forma { void render(); } class AdaptadorRectangulo implements Forma { private Rectangulo rectangulo; // composición public AdaptadorRectangulo(Rectangulo rectangulo) { this.rectangulo = rectangulo; } public void render() { rectangulo.dibujar(); // delegación } } </pre>

Como se puede notar en el código, hay extensión de rectángulo e implementación de la Forma, situaciones de múltiples herencias de Java. Por definición del Lenguaje Java, según sus especificaciones: Una clase en Java solo puede extender una única clase, aunque puede implementar múltiples interfaces (Gosling, 2014).

O sea, que ni de broma podemos hacer los siguiente:

```
class Adaptador extends ClaseA, ClaseB { } // ERROR DE COMPILACIÓN
```

De manera que, la única forma de "adaptar" dos clases de comportamiento concretas en Java es mediante composición + delegación, no herencia.

Ahora, un poco más de paciencia y detente en leer lo siguiente: Hay un autor Joshua Bloch, arquitecto de la API de colecciones de Java, quien formaliza, en el ítem 18 de "Favor composition over inheritance" que:

“Donde Java sí permite herencia simple, ésta debe evitarse entre paquetes o módulos que no han sido diseñados explícitamente para extenderse, porque romperá el encapsulamiento” (Bloch, 2018).

Ya con esto tenemos una motivación distinta a la limitación sintáctica, sino que Java conduce al diseñador hacia la composición.

1.7 El "problema raíz"

Para culminar esta sección, estimado lector, todos los patrones estructurales atacan una misma problemática: sí, pienso que todos.

Pues el objetivo va en cómo lograr que piezas de software diseñadas independientemente (o que ya existen) trabajen juntas de forma flexible, sin duplicar código ni crear dependencias rígidas (Gamma et al., 1994, cap. 4).

1.8 Variantes concretas a considerar

- Interfaces incompatibles usar **Adapter**. Abstracción e implementación que deben variar independientemente usar **Bridge**. Necesidad de tratar uniformemente objetos individuales y compuestos sería **Composite**. Necesidad de añadir funcionalidad, pero sin herencia: **Decorator**. Subsistema demasiado complejo para el cliente usar **Facade**. Demasiados objetos similares consumiendo memoria usar **Flyweight**. Necesidad de controlar el acceso a un objeto usar **Proxy**.

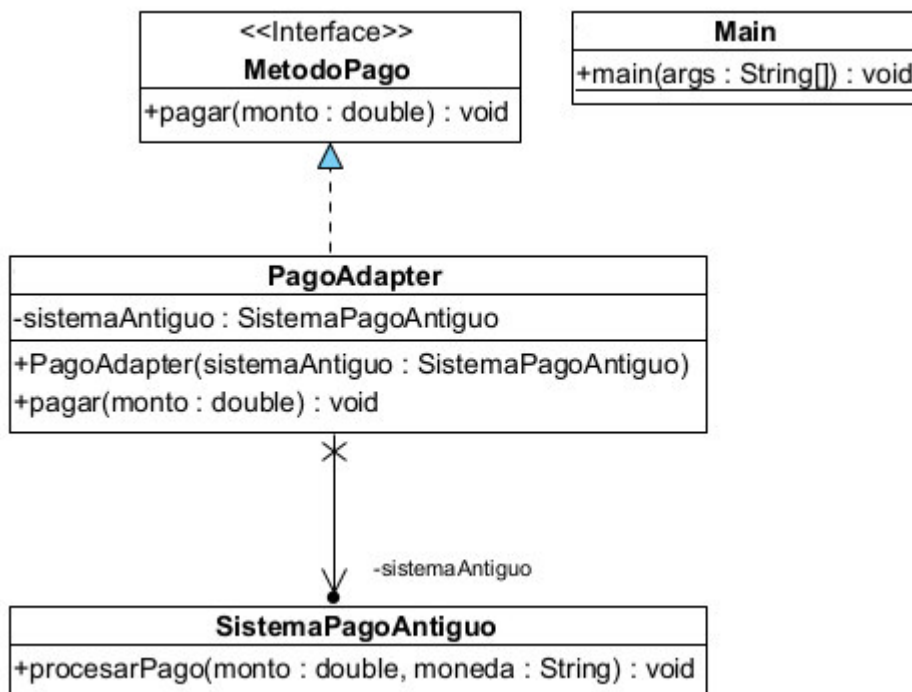
2 Adapter (Adaptador)

El patrón **Adapter** posibilita que dos componentes con interfaces incompatibles puedan trabajar juntos. El adaptador recibe las operaciones que espera el cliente y las transforma en llamadas compatibles con la clase existente. Se aplica cuando se necesita integrar código legado, una biblioteca externa o un servicio cuya interfaz no puede modificarse. En términos del GoF (Gang of Four), el patrón convierte la interfaz de una clase en otra interfaz que los clientes esperan (Metsker & Wake, 2006).

2.1 Venta de comida en una cevichería

Imaginemos una cevichería de Chimbote que tiene un sistema moderno de ventas: Sistema moderno conformado por una interfaz MetodoPago, una clase PagoAdapter, el SistemaPagoAntiguo. El sistema moderno espera: pagar(monto), pero el sistema antiguo utiliza: procesarPago(monto, moneda). No queremos modificar el sistema antiguo.

2.2 Diagrama UML Adapter



2.3 Código Adapter

Main.java

```
interface MetodoPago {
    void pagar(double monto);
}

// Sistema antiguo
class SistemaPagoAntiguo {
    public void procesarPago(double monto, String moneda) {
        System.out.println(
            "Procesando pago antiguo: S/ " + monto + " " + moneda
        );
    }
}

// Adaptador
class PagoAdapter implements MetodoPago {
    private SistemaPagoAntiguo sistemaAntiguo;
    public PagoAdapter(SistemaPagoAntiguo sistemaAntiguo) {
        this.sistemaAntiguo = sistemaAntiguo;
    }
    @Override
    public void pagar(double monto) {
        sistemaAntiguo.procesarPago(monto, "PEN");
    }
}

// Sistema de ventas de la cevichería
public class Main {
    public static void main(String[] args) {
        SistemaPagoAntiguo antiguo =
            new SistemaPagoAntiguo();
        MetodoPago pago =
            new PagoAdapter(antiguo);
    }
}
```

```
pago.pagar(85.50);  
    }  
}
```

Salida esperada:

Procesando pago antiguo: S/ 85.5 PEN

Aquí PagoAdapter permite que el sistema moderno utilice el sistema antiguo sin modificarlo.

3 Bridge (Separador)

El patrón **Bridge** separa una abstracción de su implementación para que ambas puedan trabajar independientemente. Es apropiado cuando existen dos dimensiones de variación que podrían producir una explosión de subclases si se combinan directamente. A diferencia de Adapter, que normalmente se utiliza para compatibilizar interfaces existentes, Bridge se plantea como una decisión estructural para desacoplar dos dimensiones desde el diseño.

3.1 Comida peruana y canales de atención

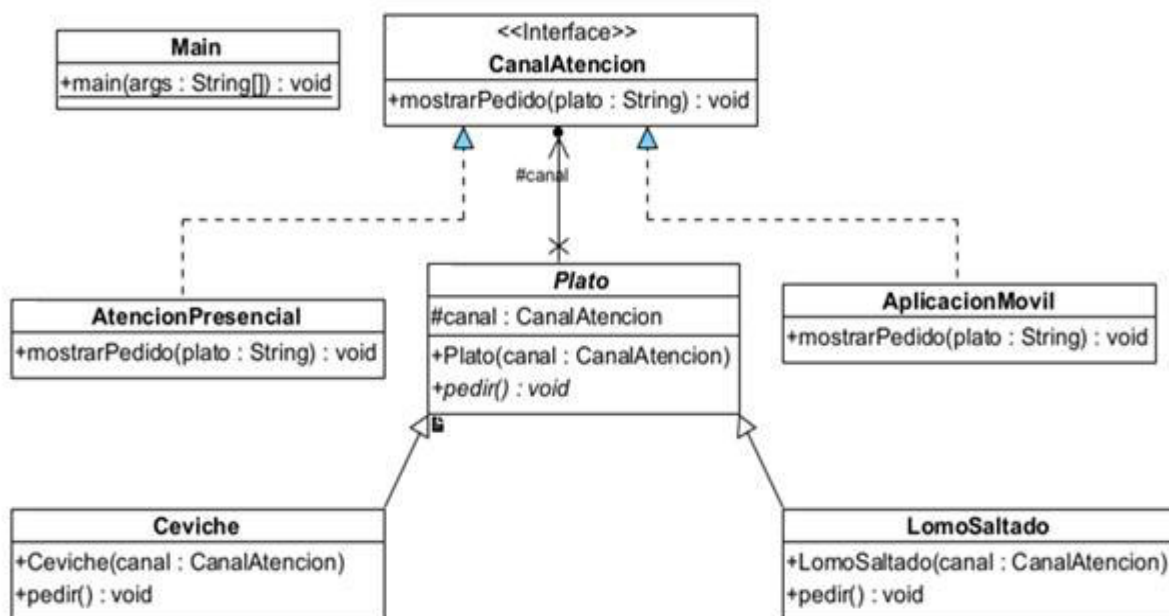
Supongamos un restaurante peruano que ofrece: comida criolla; comida marina; y desea atender mediante: atención presencial; aplicación móvil.

Si combinamos directamente ambas dimensiones tendríamos:

- ComidaCriollaPresencial ComidaCriollaApp
- ComidaMarinaPresencial ComidaMarinaApp
- ...

El Bridge separa **tipo de comida** de **canal de atención**.

3.2 Diagrama UML Bridge



3.3 Código Bridge

Main.java

```
interface CanalAtencion {
    void mostrarPedido(String plato);
}

class AtencionPresencial implements CanalAtencion {
    @Override
    public void mostrarPedido(String plato) {
        System.out.println(
            "Pedido registrado en restaurante: " + plato
        );
    }
}

class AplicacionMovil implements CanalAtencion {
    @Override
    public void mostrarPedido(String plato) {
        System.out.println(
            "Pedido registrado mediante aplicación: " + plato
        );
    }
}

// Abstracción
abstract class Plato {
    protected CanalAtencion canal;
    public Plato(CanalAtencion canal) {
        this.canal = canal;
    }
    public abstract void pedir();
}
```

```

class Ceviche extends Plato {
    public Ceviche(CanalAtencion canal) {
        super(canal);
    }

    @Override
    public void pedir() {
        canal.mostrarPedido("Ceviche peruano");
    }
}

class LomoSaltado extends Plato {
    public LomoSaltado(CanalAtencion canal) {
        super(canal);
    }

    @Override
    public void pedir() {
        canal.mostrarPedido("Lomo saltado");
    }
}

public class Main {
    public static void main(String[] args) {
        Plato ceviche =
            new Ceviche(new AplicacionMovil());
        Plato lomo =
            new LomoSaltado(new AtencionPresencial());
        ceviche.pedir();
        lomo.pedir();
    }
}

```

Salida:

Pedido registrado mediante aplicación: Ceviche peruano

Pedido registrado en restaurante: Lomo saltado

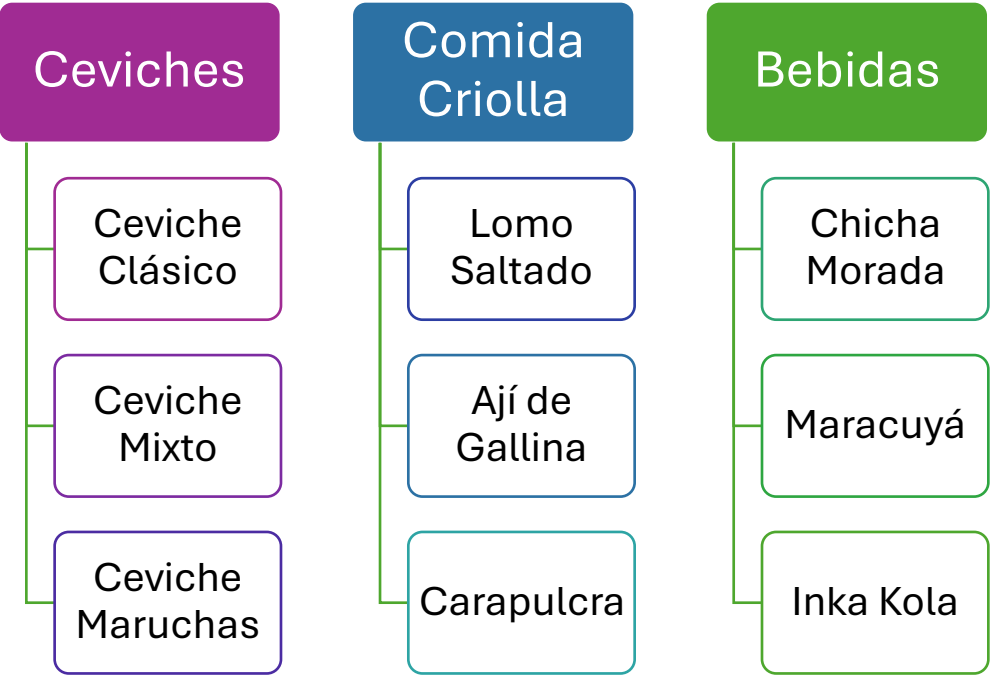
La ventaja es que podemos agregar nuevos platos y nuevos canales independientemente.

4 Composite (Compuesto)

Composite posibilita construir estructuras jerárquicas de tipo árbol y tratar de manera uniforme tanto los objetos individuales como las composiciones de objetos. El cliente puede trabajar con un componente simple o con un grupo de componentes mediante la misma interfaz. Es apropiado para estructuras de tipo **parte-todo**, como carpetas, menús, categorías y subcategorías.

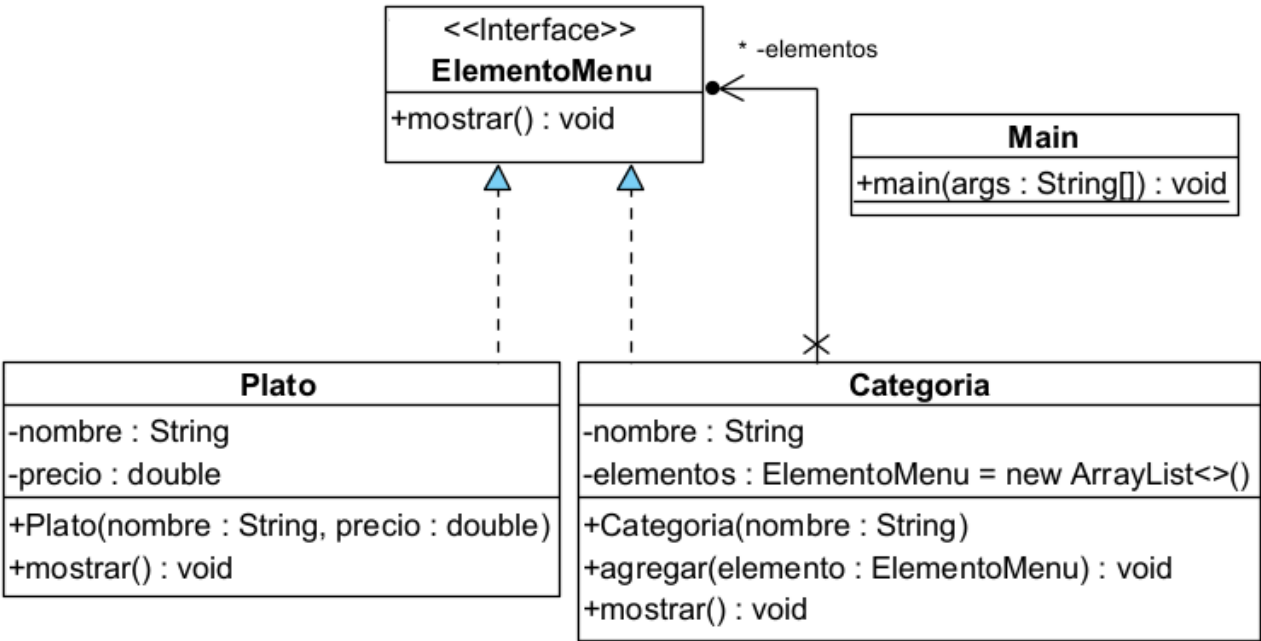
4.1 Pedido a la Carta en una cevichería

Una carta podría organizarse así:



Una categoría contiene otras categorías o productos.

4.2 Diagrama UML Composite



4.3 Código Composite

```
Main.java (composite)

import java.util.ArrayList;
import java.util.List;

interface ElementoMenu {
    void mostrar();
}

class Plato implements ElementoMenu {
    private String nombre;
    private double precio;

    public Plato(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    @Override
    public void mostrar() {
        System.out.println(
            "Plato: " + nombre +
```

```

        " - S/ " + precio

    );
}
}

class Categoria implements ElementoMenu {
    private String nombre;
    private List<ElementoMenu> elementos =
        new ArrayList<>();
    public Categoria(String nombre) {
        this.nombre = nombre;
    }
    public void agregar(ElementoMenu elemento) {
        elementos.add(elemento);
    }
    @Override
    public void mostrar() {
        System.out.println("\n[" + nombre + "]");
        for (ElementoMenu elemento : elementos) {
            elemento.mostrar();
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Categoria ceviches =
            new Categoria("Ceviches");
        ceviches.agregar(
            new Plato("Ceviche clásico", 35)
        );
        ceviches.agregar(
            new Plato("Ceviche mixto", 45)
        );
    }
}

```

```

    );

    Categoria criollos =
        new Categoria("Comida criolla");
    criollos.agregar(
        new Plato("Lomo saltado", 32)
    );

    criollos.agregar(
        new Plato("Ají de gallina", 28)
    );
    Categoria carta =
        new Categoria("CARTA PERUANA");
    carta.agregar(ceviches);
    carta.agregar(criollos);
    carta.mostrar();
}
}

```

Salida aproximada:

[CARTA PERUANA]

[Ceviches]

Plato: Ceviche clásico - S/ 35.0

Plato: Ceviche mixto - S/ 45.0

[Comida criolla]

Plato: Lomo saltado - S/ 32.0

Plato: Ají de gallina - S/ 28.0

La misma operación mostrar() funciona tanto para un plato individual como para una categoría completa.

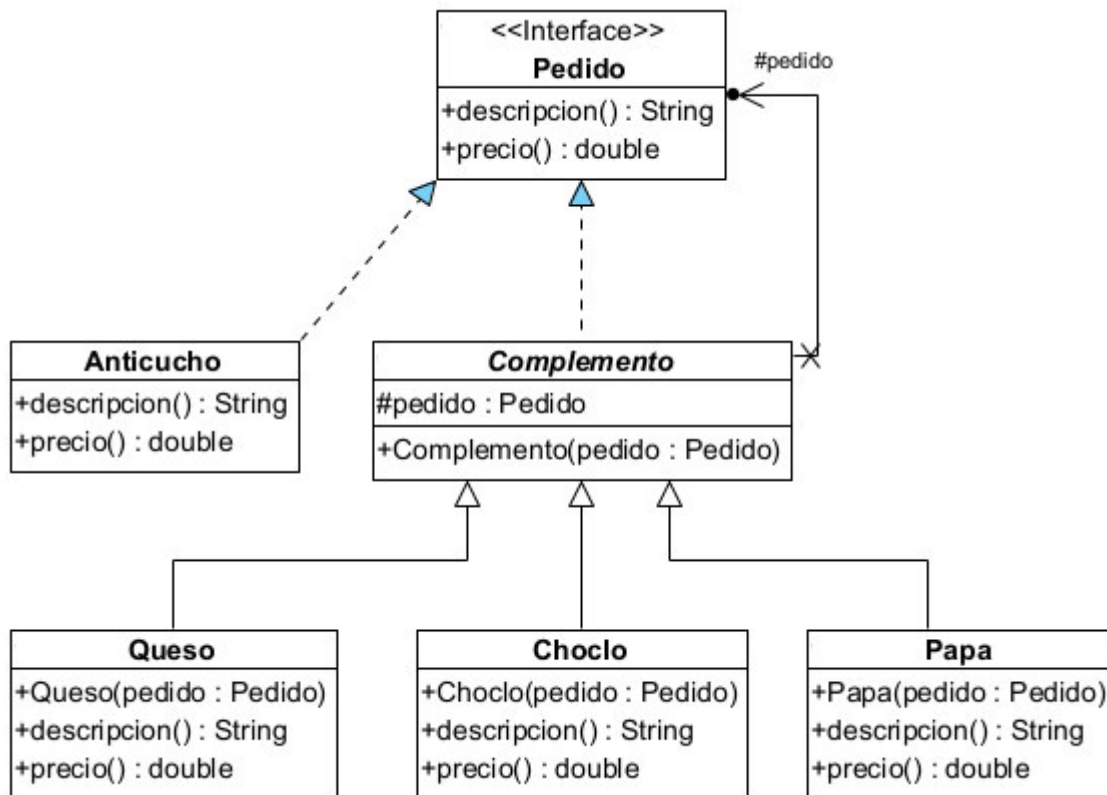
5 Decorator (Decorador)

Decorator posibilita agregar responsabilidades a un objeto dinámicamente sin modificar su clase original. El decorador mantiene la misma interfaz del objeto que envuelve y puede combinarse con otros decoradores. Constituye una alternativa flexible a crear numerosas subclases para representar todas las combinaciones posibles de funcionalidades.

5.1 Venta de anticucho con complementos

Supongamos que una aplicación vende: Anticucho + papa, + choclo, + queso. NO queremos crear de manera estática: AnticuchoConPapa o AnticuchoConPapaYChoclo o AnticuchoConPapaChocloYQueso, etc. La idea es que podemos agregar los complementos dinámicamente.

5.2 Diagrama UML Decorator



5.3 Código Decorator

Main.java (Decorator)

```
interface Pedido {
    String descripcion();
    double precio();
}

class Anticucho implements Pedido {
    @Override
    public String descripcion() {
        return "Anticucho";
    }
    @Override
    public double precio() {
        return 15.00;
    }
}

abstract class Complemento implements Pedido {
    protected Pedido;
    public Complemento(Pedido pedido) {
        this.pedido = pedido;
    }
}

class Papa extends Complemento {
    public Papa(Pedido pedido) {
        super(pedido);
    }
    @Override
    public String descripcion() {
        return pedido.descripcion() + " + papa";
    }
}
```

```

@Override

public double precio() {
    return pedido.precio() + 2;
}
}

class Choclo extends Complemento {
    public Choclo(Pedido pedido) {
        super(pedido);
    }

    @Override
    public String descripcion() {
        return pedido.descripcion() + " + choclo";
    }

    @Override
    public double precio() {
        return pedido.precio() + 3;
    }
}

class Queso extends Complemento {
    public Queso(Pedido pedido) {
        super(pedido);
    }

    @Override
    public String descripcion() {
        return pedido.descripcion() + " + queso";
    }

    @Override
    public double precio() {
        return pedido.precio() + 4;
    }
}
}

```

```

public class Main {

    public static void main(String[] args) {

        Pedido =

            new Queso(

                new Choclo(

                    new Papa(

                        new Anticucho()

                    )

                )

            );

        System.out.println(

            pedido.descripcion()

        );

        System.out.println(

            "Total: S/ " + pedido.precio()

        );

    }

}

```

Salida:

Anticucho + papa + choclo + queso

Total: S/ 24.0

El patrón permite agregar combinaciones sin multiplicar las clases.

6 Facade (Fachada)

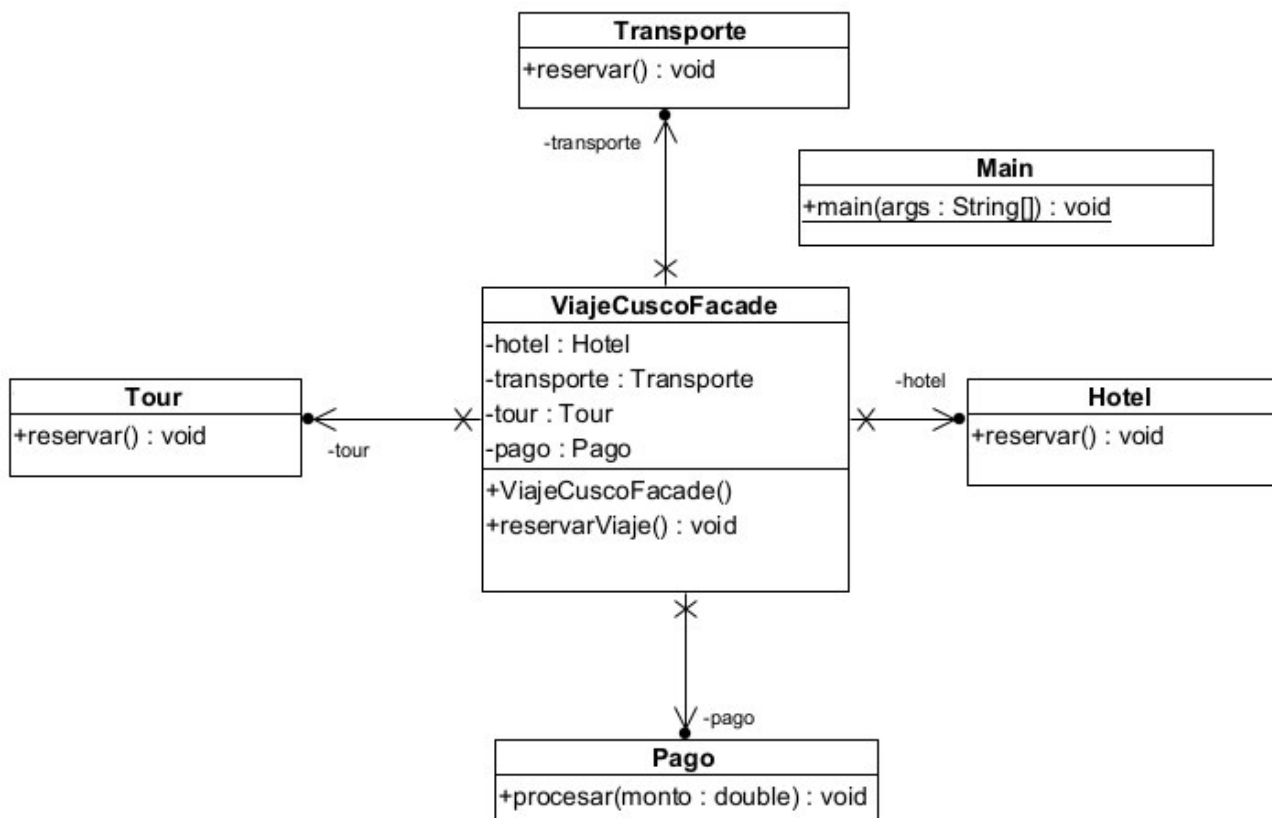
Facade proporciona una interfaz sencilla y unificada para utilizar un subsistema compuesto por múltiples clases o componentes. El cliente no necesita conocer la complejidad interna del sistema. El GoF señala que la fachada define una interfaz de nivel superior que facilita el uso de un subsistema.

6.1 Organizar un viaje turístico a Cusco

Una aplicación turística podría necesitar coordinar: ReservaHotel, ReservaTransporte, ReservaTour, Pago, Notificacion. Sin Facade el cliente tendría que controlar todos estos servicios.

Con Facade: el Cliente usa ViajeCuscoFacade y este se encarga de Hotel + Transporte + Tour + Pago.

6.2 Diagrama UML Facade



6.3 Código Facade

Main.java (Facade)

```
class Hotel {  
    public void reservar() {  
        System.out.println(  
            "Hotel reservado en Cusco."  
        );  
    }  
}  
  
class Transporte {  
    public void reservar() {  
        System.out.println(  
            "Transporte reservado."  
        );  
    }  
}  
  
class Tour {  
    public void reservar() {  
        System.out.println(  
            "Tour a Machu Picchu reservado."  
        );  
    }  
}  
  
class Pago {  
    public void procesar(double monto) {  
        System.out.println(  
            "Pago procesado: S/ " + monto  
        );  
    }  
}  
  
class ViajeCuscoFacade {  
    private Hotel;  
    private Transporte;  
    private Tour;  
    private Pago pago;
```

```

public ViajeCuscoFacade() {

    hotel = new Hotel();
    transporte = new Transporte();
    tour = new Tour();
    pago = new Pago();
}

public void reservarViaje() {
    hotel.reservar();
    transporte.reservar();
    tour.reservar();
    pago.procesar(850);
}
}

public class Main {
    public static void main(String[] args) {
        ViajeCuscoFacade viaje =
            new ViajeCuscoFacade();

        viaje.reservarViaje();
    }
}

```

Salida:

Hotel reservado en Cusco.

Transporte reservado.

Tour a Machu Picchu reservado.

Pago procesado: S/ 850.0

Pago procesado: S/ 850.0

El usuario únicamente necesita conocer: `viaje.reservarViaje()`. La complejidad queda encapsulada.

7 Flyweight (Peso ligero)

Flyweight reduce el consumo de memoria compartiendo objetos que contienen información repetida y que no cambia entre numerosas instancias lógicas. Es útil cuando una aplicación debe representar una gran cantidad de objetos pequeños que comparten características. El ejemplo clásico del GoF donde reutiliza elementos de un documento tipográfico.

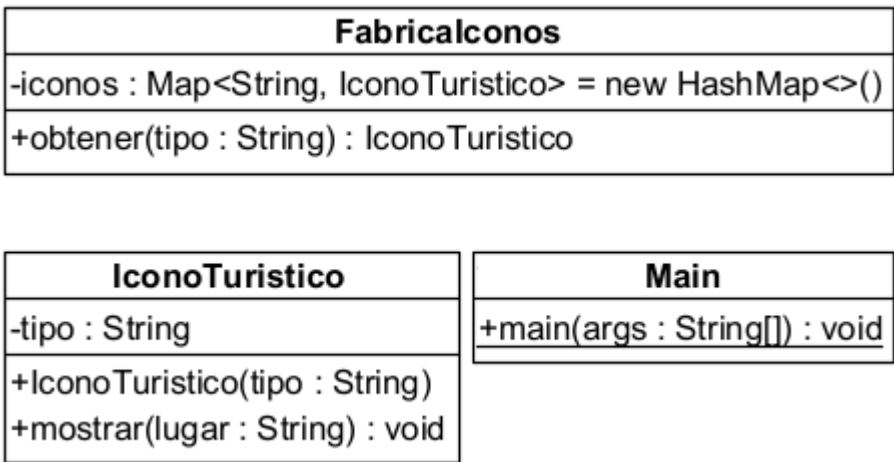
7.1 Mapa turístico del Perú

Supongamos una aplicación que muestra miles de puntos turísticos como: Machu Picchu, Chan Chan, Huaca Pucllana, Líneas de Nazca, Cañón del Colca, Parque Huascarán, etc. Muchos puntos pueden compartir información visual:

Icono = "atracción arqueológica"

No es necesario crear miles de objetos idénticos para el mismo icono.

7.2 Diagrama UML Flyweight



7.3 Código Flyweight

```
Main.java (Flywight)

import java.util.HashMap;
import java.util.Map;

class IconoTuristico {

    private String tipo;
```

```

public IconoTuristico(String tipo) {
    this.tipo = tipo;
}

public void mostrar(String lugar) {
    System.out.println(
        "Icono [" + tipo +
        "]" -> " + lugar
    );
}
}

class FabricaIconos {
    private Map<String, IconoTuristico> iconos =
        new HashMap<>();

    public IconoTuristico obtener(String tipo) {
        if (!iconos.containsKey(tipo)) {
            iconos.put(
                tipo,
                new IconoTuristico(tipo)
            );
        }
        return iconos.get(tipo);
    }
}

public class Main {
    public static void main(String[] args) {
        FabricaIconos fabrica =
            new FabricaIconos();

        IconoTuristico arqueologico =
            fabrica.obtener("Arqueológico");
        arqueologico.mostrar("Machu Picchu");
        arqueologico.mostrar("Chan Chan");
        arqueologico.mostrar("Huaca Pucllana");

        IconoTuristico arqueologico2 =
            fabrica.obtener("Arqueológico");
    }
}

```

```
System.out.println(  
    "¿Mismo objeto? " +  
    (arqueologico == arqueologico2)  
);  
}  
}
```

Salida:

Icono [Arqueológico] -> Machu Picchu

Icono [Arqueológico] -> Chan Chan

Icono [Arqueológico] -> Huaca Pucllana

¿Mismo objeto? true

El objeto IconoTuristico se comparte.

8 Proxy (representante)

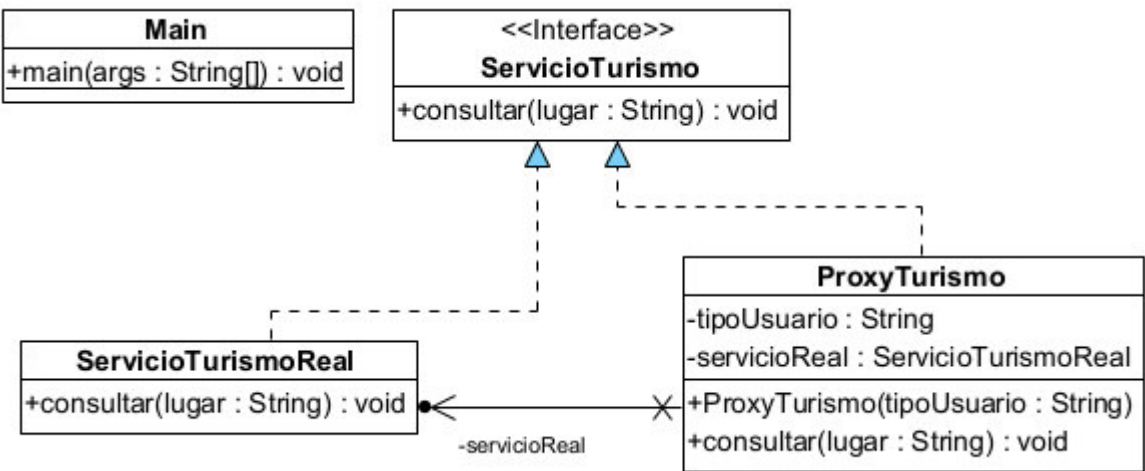
Proxy proporciona un objeto intermediario que representa a otro objeto y controla el acceso a él. Puede utilizarse, entre otros propósitos, para implementar acceso diferido, control de permisos, caché, acceso remoto o registro de operaciones. A diferencia de Adapter, el Proxy normalmente conserva la misma interfaz del objeto real.

8.1 Acceso a información turística

Imaginemos una aplicación peruana que consulta información detallada de atractivos turísticos. El acceso a la información puede ser costoso y además debe controlarse según el tipo de usuario.

Por ejemplo: Usuario accede a ProxyTurismo y así a ServicioTurismoReal. El Proxy puede verificar permisos antes de permitir el acceso.

8.2 Diagrama UML Proxy



8.3 Código Proxy

```
Main.java (Proxy)

interface ServicioTurismo {

    void consultar(String lugar);
```

```

}

class ServicioTurismoReal

    implements ServicioTurismo {

@Override

public void consultar(String lugar) {

    System.out.println(

        "Consultando información detallada de: "

        + lugar

    );

}

}

class ProxyTurismo

    implements ServicioTurismo {

private ServicioTurismoReal servicioReal;

private String tipoUsuario;

public ProxyTurismo(String tipoUsuario) {

    this.tipoUsuario = tipoUsuario;

}

@Override

public void consultar(String lugar) {

    if (!tipoUsuario.equals("ADMIN")) {

        System.out.println(

            "Acceso denegado para: "

            + tipoUsuario

        );

        return;

    }

    if (servicioReal == null) {

        servicioReal =

            new ServicioTurismoReal();

    }

    servicioReal.consultar(lugar);

}

}

```

```
public class Main {  
    public static void main(String[] args) {  
        ServicioTurismo usuario =  
            new ProxyTurismo("USUARIO");  
        usuario.consultar("Machu Picchu");  
        System.out.println();  
        ServicioTurismo administrador =  
            new ProxyTurismo("ADMIN");  
        administrador.consultar("Machu Picchu");  
    }  
}
```

Salida:

Acceso denegado para: USUARIO

Consultando información detallada de: Machu Picchu

El Proxy controla el acceso al objeto real sin modificarlo.

9 Comparación de Patrones de Diseño Estructurales

Patrón	Intención principal	Similitud clave con otros	Ventajas	Desventajas	Cuándo usarlo
Adapter	Convertir la interfaz de una clase en otra que el cliente espera	Como Facade, envuelve algo existente; pero Adapter <i>adapta</i> una interfaz, Facade <i>simplifica</i> varias	Permite reutilizar código legado sin modificarlo; bajo acoplamiento	Añade una capa extra de indirección; puede ocultar problemas de diseño subyacentes	Integrar librerías de terceros o APIs con interfaces incompatibles
Bridge	Desacoplar una abstracción de su implementación para que varíen independientemente	Similar a Adapter, pero se diseña <i>antes</i> (proactivo), no después de detectar incompatibilidad	Evita explosión de subclases; permite extender abstracción e implementación por separado	Aumenta la complejidad inicial del diseño; requiere anticipar variabilidad	Sistemas con múltiples plataformas o implementaciones (ej. drivers gráficos)
Composite	Componer objetos en estructuras de árbol para representar jerarquías parte-todo	Comparte con Decorator la composición recursiva de objetos con interfaz común	Cliente trata objetos individuales y compuestos de forma uniforme	Puede volver el diseño demasiado genérico, dificultando restricciones de tipo	Estructuras jerárquicas: árboles de UI, sistemas de archivos, organigramas
Decorator	Añadir responsabilidades a un objeto dinámicamente, sin alterar su clase	Estructuralmente idéntico a Composite (recursión), pero con propósito de extender comportamiento	Alternativa flexible a la herencia; combinaciones dinámicas de funcionalidades	Muchos decoradores pequeños dificultan la depuración y lectura del código	Añadir funcionalidades opcionales (streams de E/S, middlewares, UI widgets)

Patrón	Intención principal	Similitud clave con otros	Ventajas	Desventajas	Cuándo usarlo
Facade	Proveer una interfaz unificada y simplificada a un subsistema complejo	Como Adapter, envuelve, pero no cambia semántica sino que reduce superficie de uso	Reduce acoplamiento cliente-subsistema; mejora legibilidad y mantenibilidad	Riesgo de convertirse en "God Object" si se sobrecarga de responsabilidades	Simplificar el acceso a librerías o subsistemas complejos (ej. SDKs)
Flyweight	Compartir eficientemente objetos de grano fino para reducir uso de memoria	Comparte con Proxy el control de acceso/creación de objetos, pero orientado a optimización de recursos	Ahorro significativo de memoria en sistemas con muchos objetos similares	Introduce complejidad al separar estado intrínseco/extrínseco; puede afectar rendimiento por gestión adicional	Aplicaciones con gran cantidad de objetos similares (ej. renderizado de texto, videojuegos)
Proxy	Proveer un sustituto o representante de otro objeto para controlar el acceso a él	Similar en estructura a Decorator (misma interfaz), pero su fin es <i>controlar acceso</i> , no añadir comportamiento	Permite control de acceso, carga diferida (lazy loading), caché o seguridad	Puede introducir latencia o complejidad innecesaria si no se justifica el control de acceso	Acceso remoto, control de permisos, carga perezosa de recursos costosos

Los patrones estructurales resuelven cómo componer clases y objetos para formar estructuras más grandes, manteniendo flexibilidad y eficiencia (Gamma et al., 1994). Todos comparten el principio de **composición sobre herencia** y buscan desacoplar interfaces de implementaciones.

9.1 *Similitudes transversales*

- **Adapter, Facade, Decorator, Proxy:** envuelven un objeto existente sin modificar su código fuente.
- **Composite y Decorator:** comparten la misma estructura recursiva (interfaz común entre "hoja" y "contenedor").
- **Decorator, Proxy y Adapter:** implementan la misma interfaz que el objeto que envuelven, diferenciándose solo en la intención (extender, controlar o traducir).
- **Bridge y Adapter:** ambos desacoplan interfaces de implementaciones, pero Bridge se diseña de forma anticipada y Adapter de forma correctiva.

Una forma sencilla de recordarlos es:

**Adaptador(A)-Separador(B)-Compuesto(C)-Decorador(D)-
Fachada(F)-PesoLigero(Fw)-Controlador(P)**

10 Perspectiva moderna de los patrones de diseño estructurales

10.1 Limitaciones del patrón de diseño

Aplicar un Decorator o un Bridge donde una simple función de código hubiera bastado, conlleva a hacer Sobreingeniería, Ya Fowler advierte sobre el riesgo de que los patrones se conviertan en un fin en sí mismos en lugar de ser una herramienta par un determinado fin u objetivo(Fowler, 2004).

Cada capa de Adapter, Proxy o Decorator añade un salto de llamada, lo que dificulta el debugging y el rendimiento sobre todo si son sistemas de alta concurrencia lo cual genera demasiados “niveles de indirección”.

10.2 Patrones estructurales en arquitecturas modernas

A nivel arquitectónico podemos ir un poco más allá del nivel de clases. Por ejemplo el equivalente arquitectónico moderno de Facade sería Gateway usado en microservicios, Para Adapter se tiene la Capa Anticorrupción (ACL) del (Domain-Driven Design), para el patrón Proxy los muy usados Service Mesh o Sidecar (Kubernetes, Istio), para Decorator el uso de Middleware e Interceptors (Spring, Express). Bridge tendría su equivalente la Arquitectura Hexagonal (Ports & Adapters). Esto nos dice que los patrones no han fenecido con la POO clásica, sino que migraron: de clases a servicios (Newman, 2021).

10.3 Cuándo NO usar patrones estructurales (principio YAGNI)

Hay una expresión: "You Aren't Gonna Need It", sería mas o menos como que no te anticipes en flexibilidad si aun no ha sido solicitada (Fowler, 2004). De hecho, si solo existe una implementación concreta y no se prevé otra pues no uses Bridge. Para módulos simples, Facade no sería recomendable pues agrega indirección. Lo siguiente no es ley pero es una buena práctica: “refactoriza hacia un patrón solo la tercera vez que detectes el problema” (“Rule of Three de Martin Fowler”).

10.4 Tabla de decisión final

SITUACIÓN	USAR EL PATRÓN
¿Tengo dos interfaces incompatibles que debo conectar?	Adapter
¿Necesito que abstracción e implementación evolucionen por separado?	Bridge
¿Necesito tratar igual un objeto y un grupo de objetos?	Composite
¿Necesito añadir comportamiento sin modificar la clase?	Decorator
¿Un subsistema es demasiado complejo para el cliente?	Facade
¿Tengo miles de objetos casi idénticos consumiendo memoria?	Flyweight
¿Necesito controlar/retrasar/proteger el acceso a un objeto?	Proxy

11 Epílogo

Con el tiempo, que son algunos años, en la docencia del tema Patrones de Diseño de Software, estoy entendiendo algo: los patrones no envejecen o se deprecian como suele suceder con las funciones de los lenguajes de programación, ¿por qué?; porque nunca fueron reglas rígidas o camisas de fuerza, sino ideas, para no hacer un desarrollo de tipo espagueti o desordenado. Sino que permiten mejoras como al envolver, delegar y componer. Esa misma forma de pensar que Gamma y sus colegas describieron en 1994 sigue al parecer permaneciendo hoy, escondida en cada middleware, en cada API Gateway, en cada asunto en la que se quiere conectar un sistema heredado con uno que se piensa implementar.

Bibliografía

- Bloch, J. (2018). *Effective Java* (Third edition). Addison-Wesley.
- Eckel, B. (2007). *Thinking in Java: The definitive introduction to object oriented programming in the language of the World Wide Web* (4. ed., 11. print). Prentice-Hall.
- Fowler, M. (2004). *Inversion of Control Containers and the Dependency Injection pattern*. martinowler.com.
<https://martinfowler.com/articles/injection.html>
- Gamma, E., Helm, R., Johnson, R. E., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Gosling, J. (2014). *The Java language specification: Java SE 8 edition*. Addison-Wesley.
- Larman, C. (with Moros Valle, B., & García Molina, J.). (2003). *UML y patrones: Una introducción al análisis y diseño orientado a objetos y al proceso unificado* (2a ed). Pearson Educación.
- Martin, R. C. (2000). *Agile software development: Principles, patterns, and practices*. Prentice Hall/Pearson Education.
- Metsker, S. J., & Wake, W. C. (2006). *Design Patterns in Java™* (2nd ed). Pearson India.
- Newman, S. (2021). *Building microservices: Designing fine-grained systems* (Second edition). O'Reilly.